An Odd-Support Obstruction to Symmetric CSS Recovery

October 26, 2026

Abstract

For a binary CSS code whose X- and Z-stabilizer row spaces agree, every odd-weight zero-syndrome word is a nontrivial logical operator. Consequently no deterministic syndrome-only decoder can correct every error supported on an odd-cardinality $(a, 0)$ absorbing set. The obstruction applies to arbitrary code length and check presentations, independently of decoder dynamics. We give a general Lean proof, including the actual-check incidence bridge and recovery modulo the entire opposite stabilizer space. We then formulate the remaining even-support existence question for connected, physically inseparable, positive-dimension codes and zero-initialized syndrome-based Gallager-B. A separate one-iteration benchmark implies that question. Neither existence nor nonexistence of an even-support example is established.

1 Result and motivating question

Morris, Pllaha and Kelley ask whether every absorbing set is trapping for their syndrome-based Gallager-B decoder [4], Section VI, arXiv v2, May 10, 2024. A seven-variable simplex matrix answers the unrestricted dynamical question negatively [7]. A separate fifteen-qubit asymmetric CSS construction recovers all 128 errors on a $(7, 0)$ support, with a connected actual decoding graph, one encoded qubit and no physical product cut [8]. These are cited results, with their complete checked proofs preserved separately.

The Error Correction Zoo's *Self-dual CSS code* entry [1] supplied the terminology and discovery route to the weakly self-dual code-conversion and quotient-form references. The primary sources [5, 6], rather than the expository entry, supply the mathematical definitions and results compared below.

The natural equal-stabilizer-space restriction has an elementary but decisive parity obstruction. We use *weakly self-dual CSS* in the sense of Bravyi, Leemhuis and Terhal [5], Section 3: the binary X- and Z-stabilizer spaces are equal. This is weaker than requiring identical generating matrices and stronger than allowing a coordinate permutation to exchange unequal spaces. It is distinct from the graph-partition notion of a *symmetric stabilizer* in [4], Definition 6.

Question	Source and scope	Answer/evidence
Every absorbing set traps?	Morris–Pllaha–Kelley, v2 Section VI	No, cited simplex counterexample [7].
Can all errors on an odd $(a, 0)$ support recover in equal spaces?	Project-derived restriction; quantum success, not merely syndrome acceptance	No, Theorems 1–2.
Can a suitable even support recover in a connected equal-space code?	Project-derived Question 1, fixed decoder and physical-cut condition	Unresolved here; exact Lean interface and necessary gates.

Theorem 1 (Odd-support obstruction)**.** *Let H_X, H_Z be finite binary check matrices on n physical coordinates. Write $R_X = \operatorname{row} H_X$ and $R_Z = \operatorname{row} H_Z$. Suppose $H_X H_Z^T = 0$ and $R_X = R_Z$. Every vector of R_Z has even weight. If a has odd weight and $H_X a = 0$, then $a \notin R_Z$ and represents a nontrivial logical Z operator. In particular this applies to the indicator of any odd-cardinality $(a, 0)$ absorbing set in the actual H_X graph.*

Theorem 1 is the classical self-orthogonality parity argument applied to the decoding support. The formal contribution is a general proof with actual-matrix, induced-degree and recovery interfaces. The relevant quotient $R^\perp/R$ and its bilinear form are classical; see [6], Section 3. The complete proof and source-comparison ledgers are signposted in Appendix A; Section 6 gives the main-text reference chronology.

2 Binary spaces, incidence and the proof

Vectors lie in $\mathbb{F}_2^n$, addition is XOR, and $u \cdot v = \sum_i u_i v_i$ in $\mathbb{F}_2$. Matrices are finite ordered lists of rows; redundant and zero rows are allowed in the general theorem. The row space is the least set containing zero and every actual row and closed under XOR. Thus it is the full stabilizer space, not a selected basis or a restricted submatrix.

For a selected set A with indicator a, the degree of an actual check r into A is $|\operatorname{supp} r \cap A|$. Its parity is $r \cdot a$. Let $b_H(a)$ count actual checks of odd induced degree, counting check vertices as presented. The Lean incidence bridge proves

$$b_H(a) = 0 \quad \Longleftrightarrow \quad Ha = 0.$$

An absorbing support is nonempty and each selected variable has strictly more adjacent even-induced checks than odd-induced checks, as in [4], Definition 5. The parity obstruction needs only oddness and $b_H(a) = 0$, not the strict-majority property.

Proof of Theorem 1. Orthogonality of the actual X and Z rows extends to both row spaces by bilinearity. If $w \in R_Z = R_X$, then $w \cdot w = 0$. But $w \cdot w = |w| \pmod 2$, so w has even weight. An odd a is therefore outside R_Z. If also $H_X a = 0$, its Z operator commutes with every X stabilizer and is not a Z stabilizer. The incidence identity gives the absorbing-set application. □

Theorem 2 (No universal syndrome-only recovery)**.** *Under Theorem 1's hypotheses, fix an odd a with $H_X a = 0$. No deterministic map $D : \mathbb{F}_2^m \to \mathbb{F}_2^n$ can correct both 0 and a modulo R_Z. Hence no such map corrects every error supported on* $\operatorname{supp} a$.

Proof. Both errors produce syndrome zero. If both were corrected, then $D(0) \in R_Z$ and $a + D(0) \in R_Z$, so their sum a would belong to R_Z, contradicting Theorem 1. Both 0 and a are supported on $\operatorname{supp} a$. □

For any decoder that stops at zero syndrome with correction zero, the particular full-support error a fails logically at initialization. This is immediate logical miscorrection, not a proof of persistent syndrome mismatch or message nonconvergence. Quantum recovery and dynamical trapping must remain distinct.

3 Presentations and boundary checks

Proposition 3 (Common presentation and sectors)**.** *If $R_X = R_Z$ and the CSS condition holds, replacing H_Z by the existing H_X preserves the full binary stabilizer group, success modulo Z stabilizers, and closure of the stabilizer space under restriction to any physical cut. It leaves the actual H_X decoding graph and its message updates unchanged. For a common matrix H, a syndrome map that corrects every binary error on A corrects both components of every Pauli error on A when applied separately to the X and Z components.*

Proof. Membership in R_Z can be replaced by membership in R_X. This identifies both factors of the binary stabilizer space $R_X \times R_Z$ with $R_X \times R_X$, including membership of their coordinate restrictions. Orthogonality extends to the common presentation. The decoder uses the unchanged X check list. Finally the two corrected binary residuals belong separately to R, so their pair belongs to $R \times R$. □

This preserves the chosen Z-error decoder; it does not claim that replacing the opposite check presentation preserves a previously different X-error decoder. In the common presentation both components can use the same rule.

The hypotheses have concrete checked guards. On two coordinates the common row $(1, 1)$ has an even zero-syndrome word that is itself a stabilizer: oddness is essential. On six coordinates take $H_X = (111000)$ and $H_Z = (000111)$. The rows commute and swapping the two halves exchanges them, but the odd word 000111 is a Z stabilizer. Thus permutation-exchange symmetry does not imply the theorem's premise. Two distinct generating lists can also span the same space: identical matrices are sufficient, not necessary.

For the seven-coordinate simplex consumer, the common rows are the masks 85, 102, 51, 120, 45, 30, 75 (least-significant-bit coordinate convention). The full support is odd, every induced check has degree four, and each selected variable has four even and zero odd adjacent checks. It is absorbing and zero-syndrome, but logical in the common-matrix CSS code. The exact first-stopping definitions below reject quantum success for this word under both syndrome tests. It also supplies an anticommuting logical pair by pairing its binary vector with itself across the two sectors.

An independent integer-mask implementation checks nine fixed boundary families, including the already published asymmetric example. Under the seven equal-space controls it examines all 167 words, 13 odd zero-syndrome cases and all 1,058 possible shared outputs for those cases. This is finite semantic validation, not a substitute for the arbitrary-length theorem and not an even-support construction search.

4 The remaining existence question

We now freeze a project-derived question rather than attribute it to the source authors. Use one actual binary row list H in both CSS sectors and require $HH^T = 0$. Check vertices are the actual list positions; their ordering affects only syndrome indexing. Distinct rows are required for the existence target, although linear redundancy is allowed.

Use syndrome-based Gallager-B with variable-to-check messages initially zero. A check-to-variable message is its target syndrome bit XOR all other incident incoming messages. Hard output is the strict majority of all incoming check messages; a variable-to-check update uses the strict majority excluding that recipient check. Every tie returns zero. No channel-dependent prior vote is added. At time zero the hard output and all messages are zero. After each full iteration consider

two separately stopped runs: the hard syndrome of the hard output, or the outgoing variable-to-check parity at each check. Each run accepts at initialization for syndrome zero, otherwise at its first matching iteration, returning that iteration's hard output. The two runs may stop at different times.

Question 1. *Does there exist a finite common matrix H and nonempty proper $(a, 0)$ absorbing support A such that:*

1. *the actual bipartite Tanner graph is connected and all check rows are distinct;*
2. *there is a pair $x, z \in R^{\perp} \setminus R$ with $x \cdot z = 1$, witnessing positive logical content;*
3. *for every nontrivial physical coordinate cut C, the binary stabilizer space $R \times R$ is not closed under restriction to C;*
4. *every binary error supported on A reaches a finite first acceptance and is corrected modulo the entire R, under each of the two stopped rules?*

There is no prescribed girth, row/column degree, bounded-degree family, distance, or asymptotic LDPC condition. Four-cycles and linearly redundant checks are allowed. Product separation refers to physical coordinate partitions, not arbitrary entangling Clifford changes of variables. Membership in the full opposite stabilizer span is the quantum success criterion; merely matching a syndrome is insufficient.

Proposition 4 (Necessary gates and benchmark)**.** *Any candidate for Question 1 has even $|A|$. Every zero-syndrome error supported on A must belong to R. Requiring every supported error to succeed at initialization or after one iteration under both tests implies Question 1's recovery condition. Thus the unrestricted support-parity version of Question 1 is equivalent to its even-support version.*

Proof. A zero-syndrome input first accepts at time zero and returns zero, so it succeeds exactly when the input lies in R. Apply this to the indicator of A and use Theorem 1 to get evenness. For a nonzero syndrome, first-iteration acceptance is the first acceptance because initialization did not accept. The stated one-iteration conditions therefore give the respective first-stopped successes. The parity equivalence follows by adding the necessary evenness condition to every candidate. □

No converse from eventual success to one-iteration success is proved or assumed. Failure of a bounded search for the benchmark would leave Question 1 unresolved. Neither question is answered by the present paper. A future construction attempt should first reject supported nonstabilizer zero-syndrome words and then test complete supported-error trajectories; that attempt is outside this artifact.

5 Relation to existing results and limits

Bravyi–Leemhuis–Terhal [5], Section 3, supplies precisely the equal-space/self-orthogonality framework. Their Section 5, Corollary 1 maps any $[[n, k, d]]$ stabilizer code to a $[[4n, 2k, 2d]]$ weakly self-dual CSS code while preserving geometric locality up to constants. Its proofs control code parameters and locality. Applying it here still requires a bridge preserving the selected absorbing support, the actual Gallager-B votes and first stopping, and recovery of every error on that support. We have not established those bridges; the code conversion alone is not a checked solution of Question 1.

Haah–Hastings–Poulin–Wecker [6], Sections 3–4, studies self-orthogonal spaces, logical quotient forms and weakly self-dual code constructions for magic-state distillation. Its normal/hyperbolic distinction concerns the quotient bilinear form, not odd/even cardinality of an absorbing support. These results do not supply the particular stopped Gallager-B recovery certificate required here. Their algebraic framework supplies the relevant prior context.

Morris–Pllaha–Kelley [4], Theorem 5, distinguishes logical and degenerate zero-syndrome patterns using the induced code. An induced submatrix's row space is not automatically the entire opposite ambient stabilizer space on a proper support; we explicitly use the latter. Its Definition 3 is dynamical, whereas our obstruction is logical. Section II-B also discusses same-syndrome logical errors; syndrome equality must not be substituted for stabilizer equivalence.

Dhingra, Matthews, McDevitt and McMillon [2], Section 2 (pp. 6–8), initialize variable states with the received word and use integer or fractional unsatisfied-check bit-flip rules. Section 3 studies fixed-state absorbing patterns for those variants. Our decoder instead uses zero-initialized extrinsic syndrome messages, and quantum success is measured modulo the full opposite stabilizer space.

Morris, Pllaha and Kelley's related study [3] examines redundancy in syndrome-based quantum LDPC decoding. Its abstract describes sufficient graph conditions for convergence to the input syndrome. The distinction relevant here is between such syndrome convergence and quantum recovery: our parity obstruction concerns a nontrivial logical word sharing the identity's syndrome.

The unrestricted absorbing-implies-trapping question is answered negatively by the cited counterexample. Theorem 2 rules out universal recovery on odd zero-syndrome supports when the full X/Z stabilizer spaces agree. Question 1 asks for a connected, physically inseparable even-support example under the exact stated decoder; this paper establishes its necessary conditions and the benchmark implication, leaving its existence unresolved.

6 Reference chronology and source roles

Table 1 compares the central references and inspected milestones to this manuscript's date, October 26, 2026. The three roles are distinct: Morris–Pllaha–Kelley poses the motivating question; Error Correction Zoo supplies a discovery trail; the classical primary references supply the algebraic ingredients. The prior repository manuscripts supply the cited unrestricted and connected asymmetric witnesses.

Source	Dated milestone	Days to this manuscript
Bravyi et al. [5]	2010-04-21: First preprint	6032
Bravyi et al. [5]	2010-04-27: Inspected arXiv v2; journal year 2010	6026
Haah et al. [6]	2017-03-22: First preprint	3505
Haah et al. [6]	2017-09-29: Inspected arXiv v3	3314
Haah et al. [6]	2017-10-03: Quantum publication	3310
Morris et al. (absorbing) [4]	2023-07-26: First preprint	1188
Morris et al. (absorbing) [4]	2024-05-10: Inspected arXiv v2	899
Morris et al. (absorbing) [4]	2026-03-03: Journal publication metadata	237
Morris et al. (redundancy) [3]	2025-08-23: Author availability announcement	429
Dhingra et al. [2]	2026: Accepted manuscript; repository year only	Not asserted
Simplex manuscript [7]	2026-10-26: Dated repository manuscript; fixed cited revision	0
Connected manuscript [8]	2026-10-26: Dated repository manuscript; fixed cited revision	0
This manuscript	2026-10-26: Present dated manuscript	0

Table 1: Reference timeline. Day counts are exact for day-precision events. A source's age is not the duration of an unsolved problem.

Dates are given at their available precision: the BLT journal and the Dhingra repository record supply years, while the redundancy-paper row uses the author's dated availability announcement. Error Correction Zoo was consulted on October 26, 2026 as a discovery source. Repository manuscript rows use the dates printed on the cited documents. Investigation and packaging times are recorded separately in the reproducibility records.

A Lean interfaces, complete proofs and reproduction

The new proof closure consists of `OddSupport.lean`, `Question.lean`, `Consumers.lean` and the declaration audit. It imports only `Std` and builds cleanly with Lean 4.36.0-rc1. Every one of the 76 theorem declarations has an axiom report restricted to `propext`, `Classical.choice` and `Quot.sound`. No admitted proof or native computation oracle is used. Small Boolean cases and fixed finite consumers use kernel-evaluated `decide`; general implications use structural proofs.

A.1 Proof map and declaration links

The names below are in namespace `CSSOddSupport`, with `Question` or `Consumers` appended for their modules. Complete current and immutable declaration links follow the map.

Written proof	Exact Lean coverage
Incidence/zero syndrome	`dot_intersection_weight`, `zero_b_iff_induced_even`, `zero_b_iff_zero_syndrome`
Theorem 1	`css_spans_commute`, `equal_spans_even_stabilizers`, `odd_zero_syndrome_logical`, `odd_zero_b_logical`
Theorem 2	`cannot_correct_identity_and_odd_support`, `no_universal_supported_recovery`
Zero-stopping application	`zero_stopping_decoder_fails_odd_support`, Question's two `zero_syndrome_*_success_iff`
Proposition 3	`common_matrix_css`, `common_matrix_supported_recovery_iff`, `common_matrix_stabilizer_iff`, `common_matrix_product_cut_iff`, `common_matrix_both_sectors`
Proposition 4	Question's `full_candidate_no_supported_logical`, `full_candidate_support_even`, `full_question_iff_even`, `benchmark_implies_full`
All boundary/consumer assertions	Complete linked `Consumers.lean`; independent fixed-family replay in `verify.py`
Question 1	`Structural`, `FullCandidate`, `FullQuestion`; these are definitions, not an existence theorem.

`dot_self_weight`: current, immutable.
`css_spans_commute`: current, immutable.
`odd_zero_b_logical`: current, immutable.
`no_universal_supported_recovery`: current, immutable.
`zero_b_iff_induced_even`: current, immutable.
`common_matrix_css`: current, immutable.
`common_matrix_product_cut_iff`: current, immutable.
`common_matrix_both_sectors`: current, immutable.
`zero_syndrome_hard_success_iff`: current, immutable.
`full_question_iff_even`: current, immutable.
`benchmark_implies_full`: current, immutable.
`seven_no_decoder_corrects_all`: current, immutable.
`permutation_only_spaces_differ`: current, immutable.

The targeted follow-through ledger records the later classical variant screen and quantum redundancy bibliography. The verification manifest records every declaration, source hash and dependency. The comparison ledger and source-question screen retain access limits and question provenance. Running `build.py -lean /path/to/lean` checks a temporary clean import closure and independently replays the fixed certificate.

A.2 Dependency structure: from checks to decoder ambiguity

The two headline theorems follow this chain, with no decoder-specific convergence premise:

$$H_X H_Z^T = 0 \quad \Longrightarrow \quad R_X \perp R_Z,$$
$$R_X = R_Z \quad \Longrightarrow \quad w \in R_Z \Rightarrow w \cdot w = 0 \quad \Longrightarrow \quad |w| \equiv 0 \pmod 2,$$
$$|a| \equiv 1 \pmod 2,\ H_X a = 0 \quad \Longrightarrow \quad a \in R_X^{\perp} \setminus R_Z,$$
$$H_X a = H_X 0,\ a \notin R_Z \quad \Longrightarrow \quad \text{no shared syndrome correction succeeds on both } 0 \text{ and } a.$$

The source-to-formal path is separate: an actual check's induced degree is the weight of its intersection with the selected support, its parity is the binary dot product, and having no odd checks is exactly zero syndrome. These bridges prevent replacing ambient stabilizer membership by an induced submatrix condition. The strict-majority absorbing property is not needed after zero syndrome and oddness are known.

A.3 Finite words, full row spaces and parity

`Word n` uses the physical coordinates themselves. `Span` is the least XOR-closed set generated by the actual rows, including all redundant generators. `CSS` asserts orthogonality of actual X/Z rows, whereas `SameSpan` identifies the entire spaces.

```
abbrev Word (n : Nat) := Fin n → Bool
```

```
def dot (x y : Word n) : Bool := dotOn (List.finRange n) x y
```

```
def weight (x : Word n) : Nat := weightOn (List.finRange n) x
```

```
def Odd (x : Word n) : Prop := weight x % 2 = 1
```

```
inductive Span (rows : List (Word n)) : Word n → Prop
  | zero : Span rows zero
  | row {r : Word n} : r ∈ rows → Span rows r
  | xor {x y : Word n} : Span rows x → Span rows y → Span rows (xor x y)
```

```
def CSS (hx hz : List (Word n)) : Prop :=
  ∀ x, x ∈ hx → ∀ z, z ∈ hz → dot x z = false
```

```
def SameSpan (hx hz : List (Word n)) : Prop :=
  ∀ w, Span hx w ↔ Span hz w
```

The recursive dot-product and weight definitions, their structural parity proof and bilinearity lemmas are in the complete linked module. The checked identity below is the precise $w \cdot w = |w| \bmod 2$ interface. Orthogonality extends by induction over `Span`; equal spaces then force every stabilizer's self-dot to vanish. Here are the complete central bodies implementing that chain.

```
theorem dot_self_weight (x : Word n) :
  dot x x = decide (weight x % 2 = 1) := dotOn_self_weight _ _
```

```
theorem css_spans_commute {hx hz : List (Word n)} (hcss : CSS hx hz)
  {x z : Word n} (hX : Span hx x) (hZ : Span hz z) : dot x z = false := by
  induction hX with
  | zero => exact dot_zero_left z
  | row hr => exact row_orthogonal_span _ (hcss _ hr) hZ
  | xor hu hv ihu ihv => rw [dot_xor_left, ihu, ihv]; rfl
```

```
theorem equal_spans_self_orthogonal {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz)
  {w : Word n} (hw : Span hz w) : dot w w = false := by
  exact css_spans_commute hcss ((hsame w).mpr hw) hw
```

```
theorem equal_spans_even_stabilizers {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz)
  {w : Word n} (hw : Span hz w) : weight w % 2 = 0 := by
  have h := equal_spans_self_orthogonal hcss hsame hw
  rw [dot_self_weight] at h
  have hn : ¬ weight w % 2 = 1 := by simpa using h
  omega
```

```
theorem odd_not_stabilizer {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz)
  (a : Word n) (hodd : Odd a) : ¬ Span hz a := by
  intro ha
  have h0 := equal_spans_self_orthogonal hcss hsame ha
  have h1 := odd_dot_self a hodd
  rw [h1] at h0
  contradiction
```

A.4 Incidence and the logical-word conclusion

A zero syndrome is orthogonality against every actual decoding row. An ambient logical Z word is precisely such a word outside the entire Z span. `bValue` counts actual check-list entries of odd induced degree. The incidence equivalence and the logical conclusion use those same objects and hypotheses.

```
def ZeroSyndrome (rows : List (Word n)) (a : Word n) : Prop :=
  ∀ r, r ∈ rows → dot r a = false
```

```
def LogicalZ (hx hz : List (Word n)) (a : Word n) : Prop :=
  ZeroSyndrome hx a ∧ ¬ Span hz a
```

```
def oddChecks (rows : List (Word n)) (a : Word n) : List (Word n) :=
  rows.filter (fun r => dot r a)
```

```
def bValue (rows : List (Word n)) (a : Word n) : Nat := (oddChecks rows a).length
```

```
theorem zero_b_iff_zero_syndrome (rows : List (Word n)) (a : Word n) :
  bValue rows a = 0 ↔ ZeroSyndrome rows a := by
  simp [bValue, oddChecks, List.filter_eq_nil_iff, ZeroSyndrome]
```

```
theorem dot_false_iff_even_intersection (x y : Word n) :
  dot x y = false ↔ intersectionWeight x y % 2 = 0 := by
  rw [dot_intersection_weight]
  constructor
  · intro h
    have hn : ¬ intersectionWeight x y % 2 = 1 := by simpa using h
    omega
  · intro h
    simp [h]
```

```
theorem zero_b_iff_induced_even (rows : List (Word n)) (a : Word n) :
  bValue rows a = 0 ↔ ∀ r, r ∈ rows → intersectionWeight r a % 2 = 0 := by
  rw [zero_b_iff_zero_syndrome]
  constructor
  · intro h r hr
    exact (dot_false_iff_even_intersection r a).mp (h r hr)
  · intro h r hr
    exact (dot_false_iff_even_intersection r a).mpr (h r hr)
```

```
theorem odd_zero_syndrome_logical {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz) (a : Word n)
  (hodd : Odd a) (hzero : ZeroSyndrome hx a) : LogicalZ hx hz a := by
  exact ⟨hzero, odd_not_stabilizer hcss hsame a hodd⟩
```

```
theorem odd_zero_b_logical {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz) (a : Word n)
  (hodd : Odd a) (hb : bValue hx a = 0) : LogicalZ hx hz a := by
  exact odd_zero_syndrome_logical hcss hsame a hodd
    ((zero_b_iff_zero_syndrome hx a).mp hb)
```

A.5 Why a syndrome-only decoder cannot correct both errors

`Decoder` is an arbitrary deterministic syndrome map, not a Gallager-B-specific heuristic. `Corrects` tests the residual in the full opposite span. If identity and the odd word both succeeded, XOR closure and equal input syndromes would put the odd word in that span. The following complete proof gives the contradiction; the subsequent support application uses that both identity and the full indicator are supported on the selected set.

```
abbrev Decoder (n : Nat) := List Bool → Word n
```

```
def Corrects (hx hz : List (Word n)) (decoder : Decoder n) (e : Word n) : Prop :=
  Span hz (xor e (decoder (syndrome hx e)))
```

```
theorem cannot_correct_identity_and_odd_support {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz) (a : Word n)
  (hodd : Odd a) (hzero : ZeroSyndrome hx a) (decoder : Decoder n) :
  ¬ (Corrects hx hz decoder zero ∧ Corrects hx hz decoder a) := by
  intro ⟨h0,ha⟩
  unfold Corrects at h0 ha
  rw [xor_zero_left] at h0
  rw [zero_syndrome_equal hx a hzero] at ha
  have hr := Span.xor ha h0
  rw [xor_cancel_right] at hr
  exact odd_not_stabilizer hcss hsame a hodd hr
```

```
def Supported (e a : Word n) : Prop := ∀ i, e i = true → a i = true
```

```
def CorrectsSupported (hx hz : List (Word n)) (decoder : Decoder n) (a : Word n) : Prop :=
  ∀ e, Supported e a → Corrects hx hz decoder e
```

```
theorem no_universal_supported_recovery {hx hz : List (Word n)}
  (hcss : CSS hx hz) (hsame : SameSpan hx hz) (a : Word n)
  (hodd : Odd a) (hb : bValue hx a = 0) (decoder : Decoder n) :
  ¬ CorrectsSupported hx hz decoder a := by
  intro hall
  exact cannot_correct_identity_and_odd_support hcss hsame a hodd
    ((zero_b_iff_zero_syndrome hx a).mp hb) decoder
    ⟨hall zero (zero_supported a), hall a (self_supported a)⟩
```

The named support lemmas and XOR cancellation used above have complete bodies in the immutable source. They introduce no additional code or decoder assumptions. The theorem therefore rules out universal quantum recovery, without asserting message nonconvergence or persistent syndrome mismatch.

A.6 Common presentation, physical cuts and Pauli sectors

The presentation reductions identify span membership, not just a numerical rank. `common_matrix_css` first extends row orthogonality to the replacement presentation. The three membership equivalences then preserve recovery and the full binary stabilizer space; `common_matrix_product_cut_iff` transports closure under coordinate restriction. Complete definitions and the cut proof are at the declaration links above. The two short central reductions below show why keeping the actual X check list preserves the chosen decoder's recovery predicate and why componentwise recovery suffices for the common-matrix Pauli decoder.

```
theorem common_matrix_recovery_iff {hx hz : List (Word n)}
  (hsame : SameSpan hx hz) (decoder : Decoder n) (e : Word n) :
  Corrects hx hz decoder e ↔ Corrects hx hx decoder e := by
  exact (hsame _).symm
```

```
def CorrectsPauli (rows : List (Word n)) (decoder : Decoder n) (e : Word n × Word n) : Prop :=
  Stabilizer rows rows
    (xor e.1 (decoder (syndrome rows e.1)), xor e.2 (decoder (syndrome rows e.2)))
```

```
theorem common_matrix_both_sectors {rows : List (Word n)} (decoder : Decoder n)
  (a : Word n) (h : CorrectsSupported rows rows decoder a)
  (e : Word n × Word n) (hx : Supported e.1 a) (hz : Supported e.2 a) :
  CorrectsPauli rows decoder e := by
  exact ⟨h e.1 hx, h e.2 hz⟩
```

A.7 First stopping, the necessary gate and the benchmark

`FirstAccept` excludes every earlier accepting iteration. `Success` requires that first acceptance and its hard output to leave a stabilizer residual. The actual check, strict-majority, extrinsic-update, hard-output and outgoing-parity definitions are in `Question.lean`; Section 4 specifies their initialization, ties and two separately stopped rules. These interfaces do not replace eventual success by a finite search horizon.

```
def FirstAccept (accept : Nat → Prop) (t : Nat) : Prop :=
  accept t ∧ ∀ u, u < t → ¬ accept u
```

```
def Success (rows : List (Word n)) (accept : List Bool → Nat → Prop) (e : Word n) : Prop :=
  ∃ t, FirstAccept (accept (syndrome rows e)) t ∧
    Span rows (xor e (output rows (syndrome rows e) t))
```

```
def FullCandidate (rows : List (Word n)) (a : Word n) : Prop :=
  Structural rows a ∧ ∀ e, Supported e a → HardSuccess rows e ∧ EdgeSuccess rows e
```

```
def OneStepCandidate (rows : List (Word n)) (a : Word n) : Prop :=
  Structural rows a ∧ ∀ e, Supported e a → OneStepSuccess rows e
```

```
def FullQuestion : Prop := ∃ n, ∃ rows : List (Word n), ∃ a : Word n, FullCandidate rows a
```

```
def EvenFullQuestion : Prop := ∃ n, ∃ rows : List (Word n), ∃ a : Word n,
  FullCandidate rows a ∧ weight a % 2 = 0
```

```
def OneStepQuestion : Prop := ∃ n, ∃ rows : List (Word n), ∃ a : Word n, OneStepCandidate rows a
```

At syndrome zero, initialization is already the first acceptance and returns zero. The complete proof below reduces hard-stopped success exactly to stabilizer membership; `zero_syndrome_edge_success_iff` has the corresponding outgoing-test proof in the linked source. This is the necessary supported-logical filter, not a sufficient decoder-success criterion.

```
theorem zero_syndrome_hard_success_iff (rows : List (Word n)) (a : Word n)
  (hzero : ZeroSyndrome rows a) : HardSuccess rows a ↔ Span rows a := by
  have hs := zero_syndrome_equal rows a hzero
  constructor
  · rintro ⟨t,ht,hr⟩
    have h0 : hardAccept rows (syndrome rows a) 0 := hs
    have ht0 := first_accept_at_zero h0 ht
    simpa [ht0, output, xor_zero_right] using hr
  · intro ha
    refine ⟨0,⟨hs,?_⟩,?_⟩
    · intro u hu
      omega
    · simpa [output, xor_zero_right] using ha
```

```
def NoSupportedLogical (rows : List (Word n)) (a : Word n) : Prop :=
  ∀ e, Supported e a → ZeroSyndrome rows e → Span rows e
```

```
theorem full_candidate_no_supported_logical {rows : List (Word n)} {a : Word n}
  (h : FullCandidate rows a) : NoSupportedLogical rows a := by
  intro e he hz
  exact (zero_syndrome_hard_success_iff rows e hz).mp (h.2 e he).1
```

```
theorem full_candidate_support_even {rows : List (Word n)} {a : Word n}
  (h : FullCandidate rows a) : weight a % 2 = 0 := by
  have hz := (zero_b_iff_zero_syndrome rows a).mp h.1.zeroOddChecks
  have ha := (zero_syndrome_hard_success_iff rows a hz).mp (h.2 a (self_supported a)).1
  exact equal_spans_even_stabilizers h.1.css (identical_rows_same_span rows) ha
```

```
theorem full_question_iff_even : FullQuestion ↔ EvenFullQuestion := by
  constructor
  · rintro ⟨n,rows,a,h⟩
    exact ⟨n,rows,a,h,full_candidate_support_even h⟩
  · rintro ⟨n,rows,a,h,_⟩
    exact ⟨n,rows,a,h⟩
```

For nonzero syndrome, initialization cannot accept. Therefore a first-iteration hard match, outgoing parity match and stabilizer residual give the first stopped success under each rule; the complete `one_step_hard_success` and `one_step_edge_success` bodies establish this in `Question.lean`. The following application exposes the direction of implication. Neither existence proposition is proved here, and no converse or bounded-search nonexistence conclusion is supplied.

```
theorem one_step_candidate_full {rows : List (Word n)} {a : Word n}
  (h : OneStepCandidate rows a) : FullCandidate rows a := by
  refine ⟨h.1,?_⟩
  intro e he
  exact ⟨one_step_hard_success rows e (h.2 e he), one_step_edge_success rows e (h.2 e he)⟩
```

```
theorem benchmark_implies_full : OneStepQuestion → FullQuestion := by
  rintro ⟨n,rows,a,h⟩
  exact ⟨n,rows,a,one_step_candidate_full h⟩
```

A.8 Controls and complete supporting sources

The proof map's consumer module checks the seven-coordinate logical-failure control, the even stabilizer boundary, unequal generating lists with equal spans, and the permutation-only symmetry guard. The independent integer-mask certificate covers the fixed finite controls described in the main text. It does not establish the general theorem by enumeration or search for a construction.

Complete supporting proofs and decoder definitions are in the linked current and immutable modules and source archive. The excerpts above reproduce complete selected declarations, with typographic Unicode rendering; `excerpts.json` records their exact checked-source ranges.

References

[1] Error Correction Zoo contributors, *Self-dual CSS code*, `https://errorcorrectionzoo.org/c/self_dual_css`, inspected October 26, 2026 (discovery and terminology source).

[2] R. Dhingra, G. L. Matthews, C. McDevitt and E. McMillon, *Absorbing Sets of LDPC Codes Under a Gallager B Decoding Variant*, accepted manuscript, Virginia Tech repository (2026; year precision).

[3] K. D. Morris, T. Pllaha and C. A. Kelley, *Effect of redundancy on syndrome-based decoding for QLDPC codes*, ISTC (2025), pp. 1–5, doi:10.1109/ISTC65386.2025.11154620. Author availability announcement, August 23, 2025. Public IEEE-authored abstract at INSPIRE 2971808.

[4] K. D. Morris, T. Pllaha and C. A. Kelley, *Absorbing Sets in Quantum LDPC Codes*, arXiv:2307.14532v2 (May 10, 2024). Journal metadata: doi:10.3934/amc.2026023 (2026).

[5] S. Bravyi, B. Leemhuis and B. M. Terhal, *Majorana Fermion Codes*, arXiv:1004.3791v2 (April 27, 2010), New J. Phys. 12, 083039 (2010).

[6] J. Haah, M. B. Hastings, D. Poulin and D. Wecker, *Magic State Distillation with Low Space Overhead and Optimal Asymptotic Input Count*, arXiv:1703.07847v3 (September 29, 2017), Quantum 1, 31 (October 3, 2017).

[7] *An Absorbing Set Without Gallager-B Trapping*, paper and checked artifact, vibe-math (2026).

[8] *A Connected CSS Absorbing Set Without Gallager-B Trapping*, paper; immutable checked proof, vibe-math (2026).